Shop Sergeant — System Requirements

- **Project:** Shop Sergeant
- **Organization:** Makersmiths Makerspace
- **Scope:** Makersmiths Leesburg (MSL) — initial release
- **Status:** Phase 0 underway (Task Sheet Pipeline tooling)
- **Audience:** All stakeholders — Members, Stewards, Shop Stewards, Shop Sergeant

This document is maintained by the Jeff Irland. If you have questions/input, contact me directly via Slack (jeff.irland).

Table of Contents

1. Introduction
2. Glossary
3. Problem Statement
4. System Overview
5. Functional Requirements
6. Physical Sign-Up Sheet Requirements
7. Agentic AI and Slack Bot Requirements
8. Non-Functional Requirements
9. Success Criteria (KPIs)
10. Out of Scope
11. Appendix
    - Things Worth Considering
    - Work Tasks Data Model
    - Is it a Pipeline or Workflow?
    - Volunteer Opportunity Sign-Up Sheet

---

1. Introduction

Makersmiths' current paper-only work task volunteering process has several weaknesses. A new automated process, called Shop Sergeant, is proposed to remove much of the friction in the current process, make work task activity more visible, and gather the data for our non-

profit status. The process uses Slack as its primary communication/control medium, but can also be trigger via a few clicks on a mobile phone web form. Proposed requirements have been written here and early prototyping of the solution is underway.

1.1 Purpose and Scope

This document defines the requirements for **Shop Sergeant**, an automated task tracking system for the Makersmiths makerspace. It is written for all stakeholders — Members and Stewards who will use the system, and developers who will build it. The official version is maintained on GitHub.

This document is the starting point for the design process. It is followed by:

- `docs/specifications.md` — detailed system design, data flows, API contracts, and database schema
- `docs/implementation-plan.md` — phased development plan with file maps and code structure

This initial release targets **Makersmiths Leesburg (MSL)** only. Extension to Makersmiths Purcellville (MSP) is potentially a future milestone, but is explicitly out of scope here. The system architecture must, however, support multi-shop expansion without structural changes.

Also, it desirable for these requirements to be extensible so future Shop Sergeant features can be supported, and in addition, this solution could extended beyond it initial purpose. Such things as: reporting on location/equipment usage, shop entry/exit events, etc. There are no plans to extend this application at this time, but it is wise to design in extensibility.

1.2 Project Background

All Makersmiths Members are expected to volunteer 2 hours per month to help maintain, operate, and improve the makerspace. Today, this requirement is tracked entirely on paper: handwritten sign-up sheets are posted around the shop, Members write their name and date when they complete a task, and the sheets are periodically replaced.

This paper-only process has significant gaps — completion records are often missing or illegible, there is no cumulative history, Stewards have no visibility into whether their area's tasks are getting done, and members who regularly contribute go unrecognized.

Shop Sergeant replaces the manual bookkeeping with an automated system centered on **Slack** and **Google Sheets**, while keeping the physical sign-up sheet as the primary touchpoint for members on the shop floor.

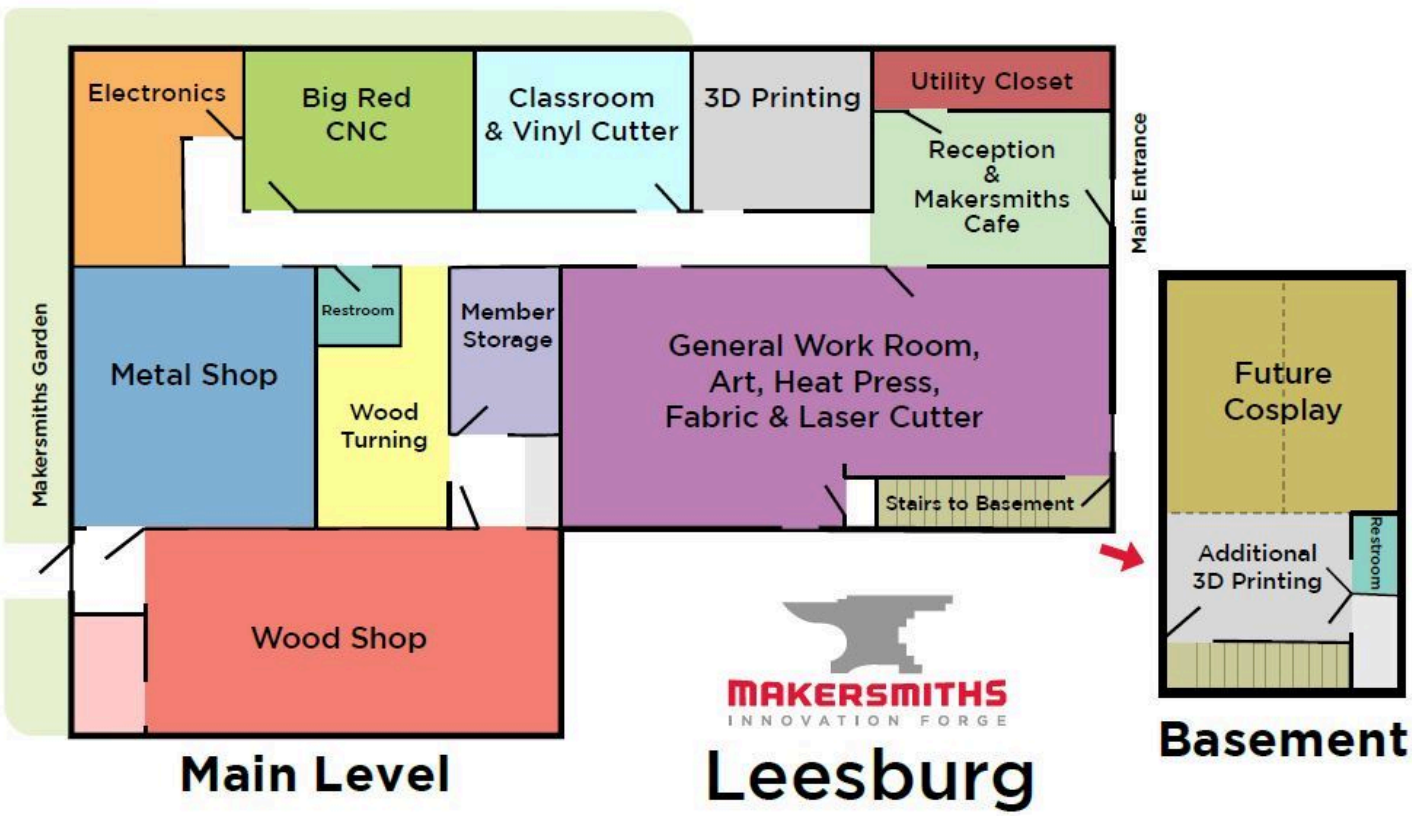
---

2. Glossary

2.1 Physical Spaces

Term	Definition	MSL Example
Shop	A physical location where Makersmiths operates. Each shop is independently managed.	Makersmiths Leesburg (MSL)
Area	A major subdivision of a shop — typically a floor, wing, or building section.	Main Floor, Exterior
Location	A specific craft or functional space within an area. Locations are the unit at which tasks and sign-up sheets are organized.	Metalshop, Laser Cutter, Wood Shop, Bathrooms

> **Diagram:** `Leesburg_Map.jpg` — Makersmiths Leesburg (MSL) Shop Floor Plan



2.2 Human Actors

These are the human participants in the Shop Sergeant system.

Actor	Who They Are	Key Responsibilities	System Access
Member	Any active Makersmiths member	Complete volunteer tasks; log completions	Log own tasks; query own history; view open tasks and public reports
Steward	A member responsible for a specific location	Define tasks for their location; monitor task completion; train members	All member actions + task add/update/delete (own location); generate sign-up sheets (own location)
Shop Steward	A member who oversees the entire shop	Monitor all areas; coordinate stewards; escalate issues	All steward actions + shop-wide access; trigger reminder blasts
Shop Sergeant	A member who manages the task volunteering process itself	Ensure the process runs smoothly; monitor KPIs; maintain the system	All shop steward actions + bot error log; audit trail; full system configuration

2.3 System Actors

These are the non-human participants in the Shop Sergeant system.

Term	What It Is	Role in the System	Interfaces With
Physical Sign-Up Sheet	Printed 8.5×11" landscape paper sheet, one per location	Primary shop-floor touchpoint for Members; hosts the task list, sign-off fields, and QR code	Member (reads/writes on it), QR Code (embedded in it)
QR Code	Machine-readable image printed on the sign-up sheet	Encodes the URL of the location's Mobile Web Form; scanning it initiates digital task logging without login or app installation	Mobile Phone (scanned by), Mobile Web Form (target URL encoded), Physical Sign-Up Sheet (embedded in)
Mobile Phone	Member's personal smartphone — no app installation required	Uses the native camera to scan a QR code; the phone's browser opens automatically to the Mobile	QR Code (scans), Mobile Web Form (browser opens to),

Term	What It Is	Role in the System	Interfaces With
		Web Form (or Slack). No app, no account, no login required for the QR method.	Slack (optional NL queries)
Mobile Web Form	Lightweight mobile-optimized web app, served when a QR code is scanned	Presents the task list for a location; collects member name and selected task; POSTs the completion record to the Agentic Workers; designed for ≤ 3 taps from scan to confirmation	Mobile Phone (rendered on), Agentic Workers (POST target for completion records)
Slack	Cloud-based team communication platform	Primary UI for all human and automated interaction; channels segment audiences and control access; Agentic Workers subscribe to events and post notifications	All human actors, Agentic Workers, Mobile Web Form (indirectly via bot)
Google Sheets	Cloud spreadsheet service accessed via Google Sheets API v4	Single source of truth for all task definitions, completion records, and member data; all pipelines read from and write to it	Agentic Workers (read/write via API), Steward / Shop Sergeant (may view directly)
Claude AI Model	Anthropic LLM (`claude-sonnet-4-6`), invoked via API by Agentic Workers	Natural language query processing — interprets plain-English questions and task completion updates from Members	Agentic Workers (invoked by)
APScheduler	Python job scheduler embedded in the bot process	Drives all time-based automation — overdue task alerts, weekly error/availability reports, monthly compliance summaries, top-	Agentic Workers (triggers actions in), Slack (posts via bot), Google Sheets (reads via bot)

Term	What It Is	Role in the System	Interfaces With
		contributor posts — without any human trigger	
Agentic Workers	Python Slack bot built with the Slack Bolt framework	Listen to Slack events; dispatch commands to the correct pipeline; invoke the Claude AI Model for NL tasks; read from and write to Google Sheets; route notifications to the correct channels	All human actors (via Slack), all Slack channels, Google Sheets, Claude AI Model, Mobile Web Form, APScheduler

2.4 System Terms

Term	Definition
Task	A discrete unit of volunteer work. Each task has a name, frequency, purpose, and instructions.
Task ID	A unique identifier for a task, formatted as `SHOP-LOCATION-NNN` (e.g., `MSL-METAL-003`). Used in all system references.
Sign-Up Sheet	A printed 8.5×11" landscape sheet listing tasks for a single location. Members write their name and date, or scan a QR code, to record completions.
QR Code	A scannable code printed on the sign-up sheet. Scanning it opens a mobile form where the member records a task completion.
Pipeline	One of four end-to-end workflows in the system: Task Database, Task Sheet, Task Capture, and Task Reporting.
Source of Truth	**Google Sheets** — all task definitions, completion records, and history live here. All other system components read from and write to Google Sheets.
Capture Method	The mechanism by which a member records a completed task. Two methods are supported: Single-Sheet QR (mobile web form) and Slack Update.

---

3. Problem Statement

The current paper-only task volunteering process has five significant weaknesses:

3.1 Incomplete Completion Records

Problem: After completing a task, members frequently do not write their name, the date, or even confirm the task as done on the sign-up sheet.

Impact: There is little to no reliable record of who did what or when. The data needed for compliance, recognition, and process improvement simply does not exist.

Requirement: The system must make recording a completion so fast and easy (target: ≤ 3 taps on a mobile device) that members do it consistently.

3.2 No Cumulative Record-Keeping

Problem: When sign-up sheets are replaced, all prior completion data is discarded. No historical record is maintained.

Impact: Makersmiths holds 501(c)(3) tax-exempt status, which may require documentation of volunteer labor. Without cumulative records, the organization cannot demonstrate member contributions over time.

Requirement: All completion records must be stored permanently in Google Sheets. Records must include member name, task ID, date, and capture method used.

3.3 Stewards Have No Monitoring Tools

Problem: Stewards are responsible for their location's tasks but have no way to see which tasks are overdue, which are being completed regularly, or which are being ignored.

Impact: Problems in a location are only discovered when something breaks or a safety issue arises — not proactively.

Requirement: Stewards must be able to query task status for their location at any time via Slack. The system must automatically alert Stewards when tasks are overdue.

3.4 Periodic Tasks Have No Scheduling

Problem: Some tasks must be performed on a regular schedule (weekly, monthly, quarterly) but there is no mechanism to track whether they have been done on time.

Impact: Periodic maintenance gets forgotten. Equipment degrades. Safety risks accumulate.

Requirement: Tasks must have an associated frequency. The system must track last-completion dates and generate alerts when periodic tasks are overdue.

3.5 Top Contributors Go Unrecognized

Problem: Members who consistently go above and beyond their 2-hour requirement are not identified or thanked, because no records exist.

Impact: Highly engaged Members receive no recognition. This reduces motivation and community goodwill.

Requirement: The system must generate periodic reports identifying top contributors so the Shop Sergeant and Stewards can recognize them.

---

4. System Overview

4.1 Architecture Summary

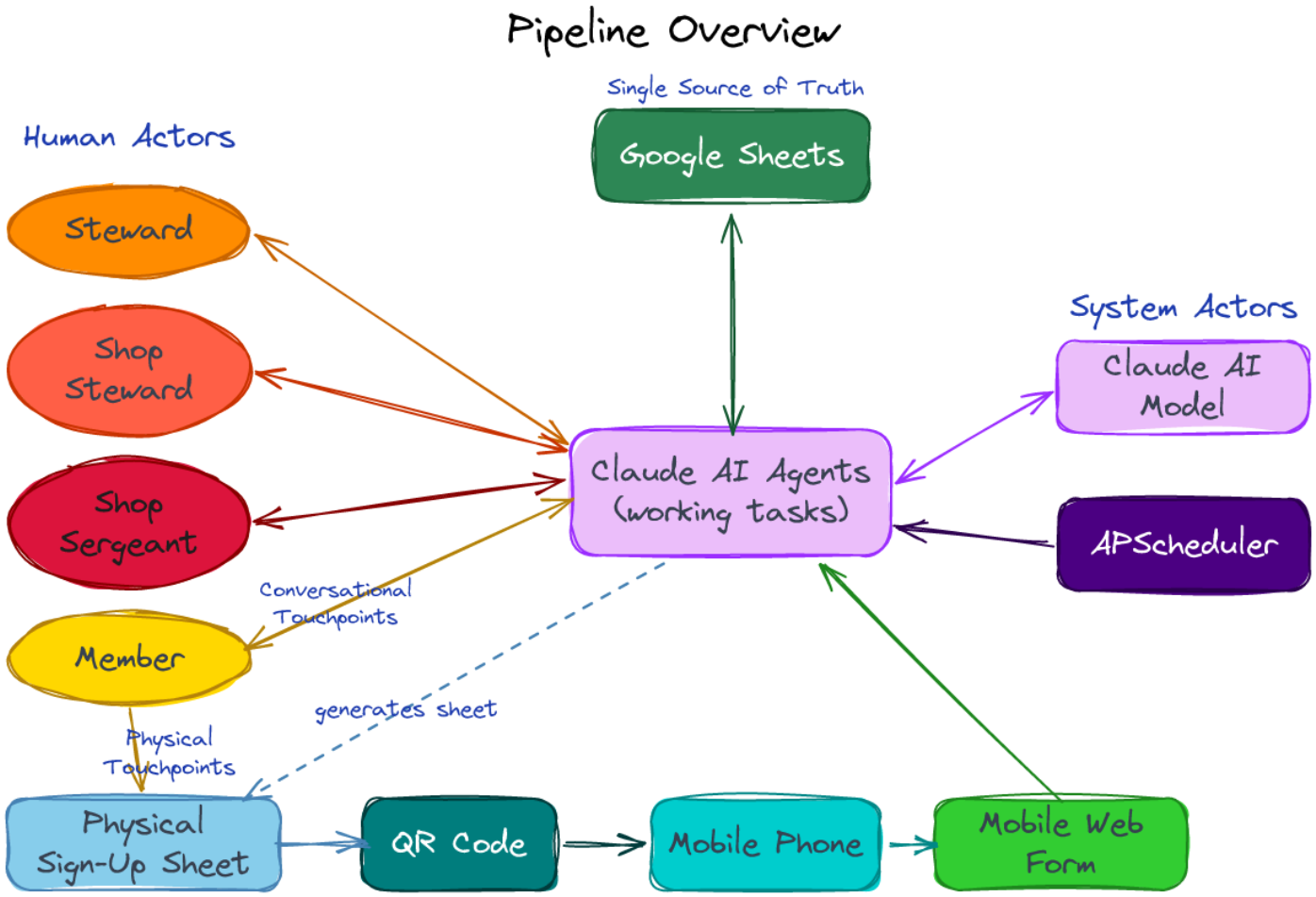
Shop Sergeant is built around four pipelines, each handling a distinct part of the volunteer task workflow:

Pipeline	What It Does
Task Database	Maintains the master list of all tasks in Google Sheets; supports add, update, and delete
Task Sheet	Generates printable sign-up sheets from the task database and posts them around the shop
Task Capture	Records task completions from Members via Single-Sheet QR → mobile web form (Method 1) or Slack Update in `#shop-queries` (Method 2)
Task Reporting	Generates reports on task completion, Member participation, and location health

Google Sheets is the system's single source of truth. All pipelines read from and write to it. **Slack** is the primary user interface for both members and administrators. Physical sign-up sheets serve as the member-facing touchpoint on the shop floor.

Human actors (Member, Steward, Shop Steward, Shop Sergeant) are defined in §2.2. System actors (Physical Sign-Up Sheet, QR Code, Mobile Phone, Mobile Web Form, Slack, Google Sheets, Claude AI Model, APScheduler, Agentic Workers) are defined in §2.3.

> **Diagram:** `pipeline-overview.excalidraw` — end-to-end pipeline overview



4.2 Technology Stack

Component	Technology	Role
Bot framework	Python 3.11+ with Slack Bolt	Listens to Slack events; dispatches commands to the appropriate pipeline
AI agent	Claude API (`claude-sonnet-4-6`)	Natural language query handling; Slack Update task completion processing
Data store	Google Sheets API v4	Authoritative storage for all tasks and completion records
Sheet generation	Jinja2 + wkhtmltopdf	Renders HTML sign-up sheet templates to PDF for printing
QR codes	`qrcode[pil]` Python library	Generates QR codes embedded in sign-up sheets

Component	Technology	Role
Scheduling	APScheduler	Sends recurring reminders; monitors overdue periodic tasks
Testing	pytest	Unit and integration tests for all pipeline components

4.3 Current State

The current state of the project, a prototyping step that will be called Phase 0, has implemented the tooling for the **Task Sheet Pipeline** process and passing automated tests.

- `scripts/signup-sheet.py` — CLI tool that renders a sign-up sheet HTML from a YAML task file
- `scripts/generate-signup-sheet-template.py` — generates the reusable Jinja2 HTML template
- `scripts/yaml-to-sheets.py` — converts YAML task definitions to Excel for Google Sheets import
- `scripts/parse-tasks.py` and `parse-opp-tasks.py` — YAML to Markdown converters for review

Phase 0: Sign-Up Sheet Prototype

Using the above scripts, the project is ready for prototyping of the Volunteer Opportunities sign-up sheet with a limited number of stewards and members. No mobile phone, Slack bot, or Google Sheets integration exists yet.

The objective is to assess the easy of use of different sign-up sheet formats with a mobile phone. Will the user know how to use their mobile device and how error prone will that use be?

Phase 0: Task Capture Prototype

The next prototyping step will cover both capture methods: the mobile phone QR code scan opening a mobile web form (Method 1) and the Slack Update flow in `#shop-queries` (Method 2). Some small parts of the Slack bot or Google Sheets integration may be simulated.

The objective is to assess the ease of use of each capture method and determine which members prefer.

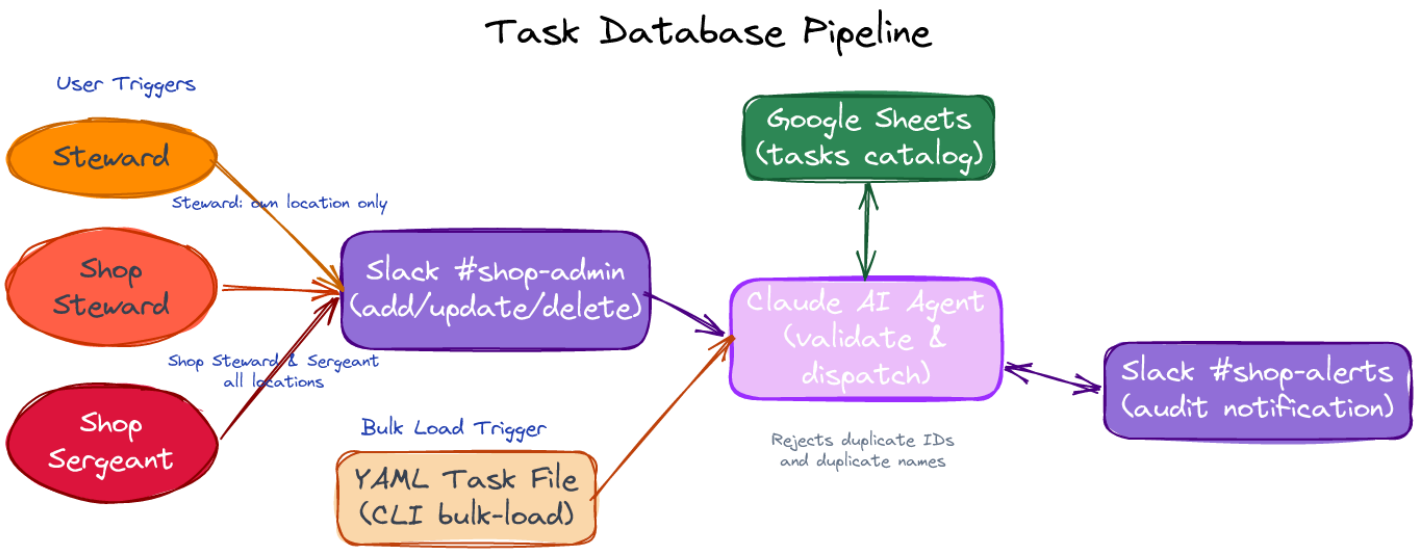
---

5. Functional Requirements

5.1 Task Database Pipeline

Purpose: Maintain the authoritative catalog of all volunteer tasks in Google Sheets.

> **Diagram:** `task-database-pipeline.excalidraw`



Inputs: YAML task definition files; Slack commands in `#shop-admin`

Outputs: Updated Google Sheets task catalog; Slack notifications in `#shop-alerts`

Requirements:

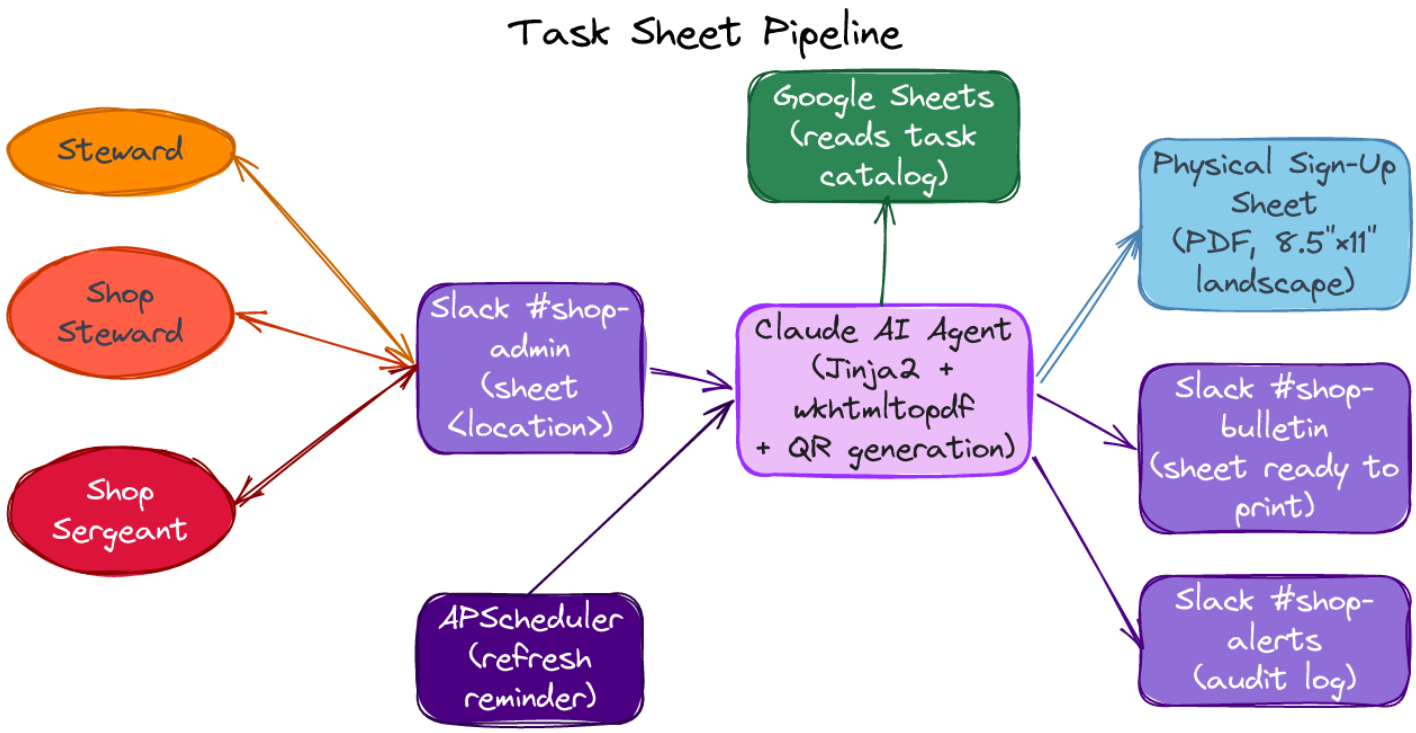
#	Requirement
DB-1	Google Sheets is the single source of truth for all task definitions and completion records
DB-2	Task definitions may be bulk-loaded from YAML files using provided scripts
DB-3	Individual tasks may be added, updated, or deleted via Slack commands in `#shop-admin`
DB-4	Only stewards (for their own location), shop stewards, and the shop sergeant may modify tasks
DB-5	Every task must have a unique Task ID (`SHOP-LOCATION-NNN`), a name, a frequency, and a location

#	Requirement
DB-6	The system must reject duplicate Task IDs and duplicate task names within the same location
DB-7	Every modification to the task catalog must trigger a notification in `#shop-alerts`

5.2 Task Sheet Pipeline

Purpose: Generate printable sign-up sheets from the task database and notify stewards when new sheets are needed.

> **Diagram:** `task-sheet-pipeline.excalidraw`



Inputs: Task YAML files (exported from Sheets); Slack commands in `#shop-admin`; optional logo and custom reference doc

Outputs: PDF sign-up sheets ready for printing; Slack notifications in `#shop-bulletin` and `#shop-alerts`

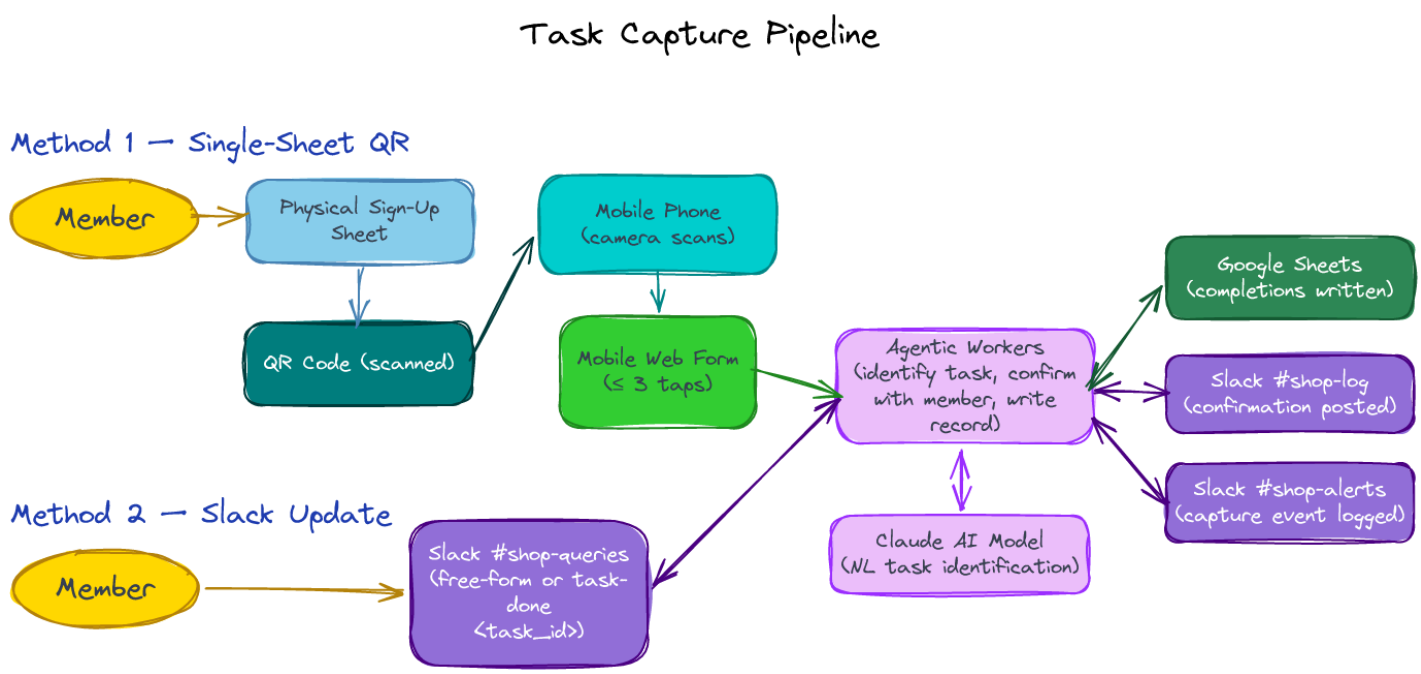
Requirements:

#	Requirement
TS-1	The system must generate a sign-up sheet for any single location or all locations in a shop
TS-2	Only stewards (own location), shop stewards, and the shop sergeant may generate sign-up sheets
TS-3	Sheets must be generated via Slack command in `#shop-admin` or via CLI script
TS-4	`#shop-bulletin` must be notified when new sheets are ready for printing
TS-5	`#shop-alerts` must be notified each time a sheet is generated
TS-6	The Slack bot must periodically remind stewards and the shop steward when task changes make a sheet refresh advisable

5.3 Task Capture Pipeline

Purpose: Record task completions from members via mobile device, with minimum friction.

> **Diagram:** `task-capture-pipeline.excalidraw`



Inputs: QR code scan from member's mobile device; Slack message in `#shop-queries`

Outputs: Completion record written to Google Sheets; confirmation message in `#shop-log`; notification in `#shop-alerts`

Two capture methods are supported.

Method	Description
Method 1 — Single-Sheet QR	A single QR code appears on each sign-up sheet. Scanning opens a mobile web form where the Member selects their name and the task completed, then confirms. The form posts to the bot, which logs to Sheets with `source=QR-form`.
Method 2 — Slack Update	Member posts a task completion in `#shop-queries` using free-form text (e.g. "I just finished cleaning the metalshop floor") or the `task-done <task_id>` command. Task ID is accepted when provided but not required. The bot identifies the task, confirms with the member, and logs the completion to Sheets with `source=Slack`.

Requirements:

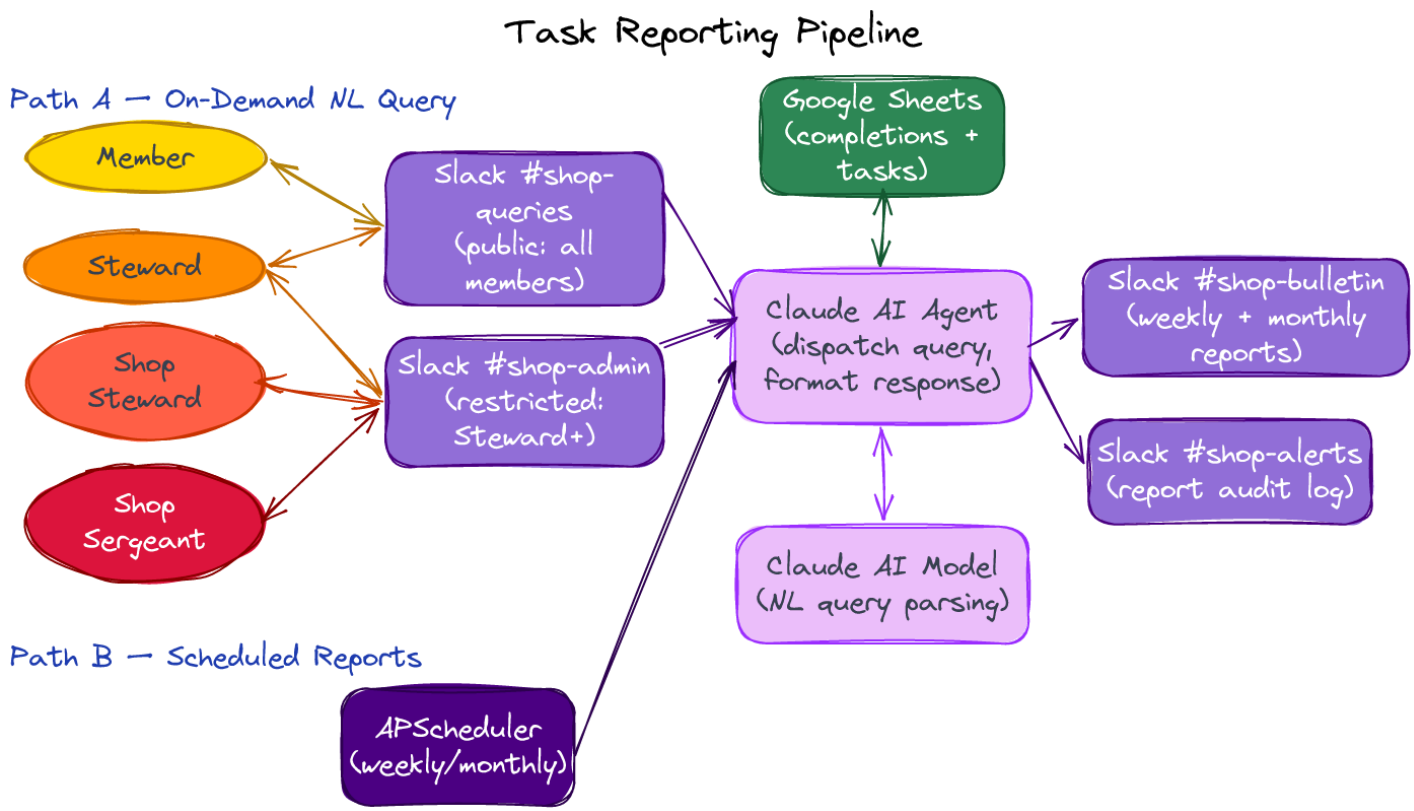
#	Requirement
TC-1	Every completion record must include: Member name, Task ID, completion date, and capture method (`QR-form` or `Slack`)
TC-2	Completion records must be written to Google Sheets immediately upon submission
TC-3	A confirmation message must appear in `#shop-log` for every logged completion
TC-4	`#shop-alerts` must be notified whenever a capture event is processed
TC-5	The Method 1 (QR) capture flow must require ≤ 3 taps on a mobile device from QR scan to confirmation
TC-6	Method 1 (QR) must not require Members to create an account or install an app; Method 2 (Slack) assumes the member has an existing Slack account
TC-7	The bot must accept task completions in `#shop-queries` via free-form natural language text or the `task-done <task_id>` command
TC-8	When no Task ID is provided in a Slack Update, the bot must identify the task via natural language processing or present a task list for member selection

#	Requirement
TC-9	The bot must confirm the identified task with the member before writing the completion record to Google Sheets

5.4 Task Reporting Pipeline

Purpose: Generate reports on task completion, Member participation, and location health, accessible via Slack.

> **Diagram:** `task-reporting-pipeline.excalidraw`



Inputs: Natural language queries in `#shop-queries` or `#shop-admin`; scheduled report triggers

Outputs: Inline Slack report responses; `#shop-alerts` notification on report generation

Report types:

Report	Description	Access
Member report	Tasks completed by a specific Member; compliance status	Member (own only), Stewards+ (any Member)

Report	Description	Access
Location report	Task completion status for a specific location; overdue tasks	All Members (public data)
Shop report	Aggregate compliance, overdue tasks, top contributors, errors logged, availability across all locations	All Members (public data)
Compliance report	Which Members have/have not met the 2hr/month requirement	Shop Steward and Shop Sergeant only

Requirements:

#	Requirement
TR-1	Members may request reports via natural language in `#shop-queries` (public data only)
TR-2	Stewards, Shop Stewards, and Shop Sergeants may request any report type via `#shop-admin`
TR-3	The bot must respond to report requests inline in Slack
TR-4	`#shop-alerts` must be notified when any report is generated
TR-5	A monthly summary report must be automatically posted to `#shop-bulletin`
TR-6	A monthly top-contributors report must be automatically posted to `#shop-bulletin`
TR-7	A weekly report of errors logged, availability across all locations posted to `#shop-bulletin`

---

6. Physical Sign-Up Sheet Requirements

The sign-up sheet is the primary Member-facing touchpoint. Most Members will interact with the system only through this physical sheet.

6.1 Format

Property	Requirement
Paper size	8.5×11 inches, landscape orientation
One sheet per location	Each location (e.g., Metalshop, Laser Cutter) gets its own sheet
Print compatibility	Must be legible when printed in black and white
Logo	Makersmiths logo in the sheet header
Header info	Location name, Steward name, date generated

6.2 Columns

Each task row on the sign-up sheet must include:

Column	Content
Task Name	Human-readable task description
Frequency	How often the task should be done (e.g., Weekly, Monthly)
Supervisor Required	Yes/No — whether a Steward must be present
Completion Date	Blank — Member writes the date by hand (fallback if QR not used)
Member Name	Blank — Member writes their name by hand (fallback if QR not used)
QR Code	Scannable code for digital task logging

6.3 Task Display Rules

- Tasks are grouped by location
- Tasks that are TBD or have no defined work are silently omitted from the sheet
- Task rows are ordered by frequency (most frequent first) within a location

6.4 QR Code Placement

The QR code placement depends on the capture method selected:

- **Method 1 (Single-Sheet QR):** One QR code in the sheet header or footer, covering the entire location

- **Method 2 (Slack Update):** No QR code required — member interacts with the bot via `#shop-queries`

---

7. Agentic AI and Slack Bot Requirements

Shop Sergeant uses a **custom Slack bot** built with the Slack Bolt framework (Python) and the Claude API. This is distinct from Anthropic's "Claude in Slack" product — the custom bot can read from and write to Google Sheets, execute task management commands, and send structured notifications.

The Claude AI model powers **natural language query handling** — members and stewards ask questions in plain English, and the bot interprets free-form Slack Update messages to identify the intended task.

7.1 Action Definitions

- **Log task completion (QR or Slack):** Submit a record that a specific task has been completed. Triggered by scanning a QR code and submitting the mobile web form (Method 1) or by posting a completion message in `#shop-queries` via free-form text or `task-done <task_id>` command (Method 2). The bot writes a completion record to Google Sheets with member name, Task ID, date, and capture method (`QR-form` or `Slack`).
- **Query own task history:** Ask the bot to retrieve the requesting member's own completion records — how many tasks completed, which ones, and whether the 2hr/month requirement has been met.
- **Query any Member's task history:** Ask the bot to retrieve completion records for a specific other member by name or Slack handle. Restricted to protect member privacy.
- **Query open tasks / location status:** Ask the bot which tasks are currently open (not yet completed within their scheduled period) for a given location or across the shop. Returns public task data only.
- **Add / update / delete a task:** Instruct the bot to create a new task, modify an existing task's fields (name, frequency, instructions, supervisor requirement), or remove a task from the catalog. Changes are written to Google Sheets and trigger a `#shop-alerts` notification.
- **Generate / print sign-up sheet:** Instruct the bot to render a new printable sign-up sheet PDF for a given location (or all locations) from the current task catalog. The bot posts the sheet and notifies `#shop-bulletin`.

- **Request location or shop report:** Ask the bot to generate a summary report — either for a specific location (task completion status, overdue tasks) or shop-wide (aggregate compliance, top contributors). Public-data reports are available to all Members; the compliance report (which Members have/have not met the 2hr/month requirement) is restricted to Shop Steward and Shop Sergeant.
- **Trigger overdue reminder blast:** Instruct the bot to immediately send an overdue-task reminder message to `#shop-bulletin` for all locations with tasks past their scheduled period, rather than waiting for the next scheduled reminder.
- **View bot errors / audit log:** Read the `#shop-alerts` channel, which contains every significant system event — task CRUD operations, capture events, report generation, and bot errors. Provides the full audit trail for the system.
- **Suspend / restore actions — area or location scope:** Disable or re-enable any bot action (e.g., task logging, sign-up sheet generation) for a specific area or location within the shop. While suspended, the bot rejects affected actions for that scope and posts a notice to `#shop-alerts`. Used by Shop Stewards to lock down a location during an incident, equipment removal, or process review. Shop Stewards may only suspend/restore actions within their own area; Shop Sergeants may act on any area or location.
- **Suspend / restore actions — shop scope:** Disable or re-enable any bot action across the entire shop. While suspended, the bot rejects the affected actions shop-wide and posts a notice to `#shop-alerts`. Reserved for Shop Sergeant use only — intended for system-wide incidents, emergency shutdowns, or major process changes.

7.2 Slack Channel Structure

Channel	Access	Who Posts	Purpose
`#shop-log`	All Members (read), Bot (write)	Bot only	Task completion confirmations; one message per logged completion
`#shop-bulletin`	All Members (read), Bot (write)	Bot only	Announcements: new sign-up sheets available, overdue reminders, monthly summaries, top contributors
`#shop-queries`	All Members	Members + Bot	Natural language queries about tasks, Member history, location status, and public reports
`#shop-admin` 🔒	Stewards, Shop Steward, Shop	Admins + Bot	Task add/update/delete, sign-up sheet generation, restricted report requests,

Channel	Access	Who Posts	Purpose
	Sergeant		compliance queries
`#shop-alerts` 🔒	Shop Sergeant	Bot only	Bot errors, audit trail, system event log — every significant system action generates an entry here

7.3 Permission Model

The following table defines what each actor may ask the bot to do, and where:

Action	Member	Steward	Shop Steward	Shop Sergeant	Channel
Log task completion (QR or Slack)	✅	✅	✅	✅	web form / `#shop-queries`
Query own task history	✅	✅	✅	✅	`#shop-queries`
Query any Member's task history	❌	❌	✅	✅	`#shop-admin`
Query open tasks / location status	✅	✅	✅	✅	`#shop-queries`
Add / update / delete a task	❌	✅ (own location)	✅	✅	`#shop-admin`
Generate / print sign-up sheet	❌	✅ (own location)	✅	✅	`#shop-admin`
Request location or shop report	✅ (public data)	✅	✅	✅	`#shop-queries` / `#shop-admin`
Trigger overdue reminder blast	❌	❌	✅	✅	`#shop-admin`

Action	Member	Steward	Shop Steward	Shop Sergeant	Channel
View bot errors / audit log	❌	❌	✅	✅	`#shop-alerts`
Suspend / restore actions — area or location scope	❌	✅	✅	✅	`#shop-admin`
Suspend / restore actions — shop scope	❌	✅	✅	✅	`#shop-admin`

> **Note:** Steward permissions are scoped to their own location. A Metalshop steward cannot modify Laser Cutter tasks, but Stewards are given permission to do Suspend / restore actions so they can take action in the event of a problem.

7.4 Agent Capabilities

The Claude-powered bot agent must be able to:

Capability	Description
Natural language query	Parse plain-English questions from `#shop-queries` and respond with data from Google Sheets. Example: "What tasks are still open in the metalshop?"
Task completion logging	Accept completion events from QR/form submissions (Method 1) or Slack Update messages in `#shop-queries` (Method 2); confirm task identity with member; write structured records to Google Sheets
Task management	Execute add, update, and delete operations on Google Sheets in response to Slack commands from authorized users
Report generation	Query Google Sheets and return formatted reports inline in Slack — member, location, shop, and compliance report types
Scheduled reminders	Use APScheduler to send periodic reminders to `#shop-bulletin`: overdue task alerts, monthly compliance summaries, top-contributor recognition
Notification routing	Route system event notifications to the correct channel (`#shop-log`, `#shop-bulletin`, or `#shop-alerts`) based on event type

7.5 Example Interactions

The following examples illustrate how actors will interact with the Slack bot.

Example 1: Member logs a completed task (Method 1 — Single-Sheet QR)

> Member scans QR code on the Metalshop sign-up sheet. A mobile web form opens showing the Metalshop task list. Member taps "Clean and sweep metalshop floor" and taps "Submit."

Bot posts to `#shop-log`:

```
✅ Task logged
Member:  @alex_johnson
Task:    MSL-METAL-003 — Clean and sweep metalshop floor
Date:    2026-04-13
Method:  QR form
```

Example 2: Member logs a completed task (Method 2 — Slack Update, free-form)

In `#shop-queries`:

> **@alex_johnson:** I just finished cleaning the metalshop floor

> **@shop-sergeant-bot:** Got it — did you mean **MSL-METAL-003 — Clean and sweep metalshop floor**? Reply `yes` to confirm or tell me more.

> **@alex_johnson:** yes

Bot posts to `#shop-log`:

```
✅ Task logged
Member:  @alex_johnson
Task:    MSL-METAL-003 — Clean and sweep metalshop floor
Date:    2026-04-13
Method:  Slack
```

Example 3: Member logs a completed task (Method 2 — Slack Update, task-done command)

In `#shop-queries`:

> **@alex_johnson:** task-done MSL-METAL-003

> **@shop-sergeant-bot:** Confirm: **MSL-METAL-003 — Clean and sweep metalshop floor** completed today? Reply `yes` to log it.

> **@alex_johnson:** yes

Bot posts to `#shop-log`:

```
✅ Task logged
Member:  @alex_johnson
Task:    MSL-METAL-003 — Clean and sweep metalshop floor
Date:    2026-04-13
Method:  Slack
```

Example 4: Member queries own task history

In `#shop-queries`:

> **@alex_johnson:** How many tasks have I done this month?

> **@shop-sergeant-bot:** You've completed 3 tasks in April 2026:
>
> - Apr 2 — MSL-METAL-003 Clean and sweep metalshop floor
> - Apr 7 — MSL-LASER-002 Clean laser cutter bed and optics
> - Apr 11 — MSL-METAL-005 Check and top off cutting fluids
>
> You've met your 2-hour volunteer requirement for April.

Example 5: Shop steward queries another member's task history

In `#shop-admin`:

> **@mike_shopsteward:** How many tasks has @dave_m completed this month?

> **@shop-sergeant-bot:** @dave_m has completed 2 tasks in April 2026:
>
> - Apr 3 — MSL-WOOD-001 Clean and sweep wood shop floor
> - Apr 9 — MSL-LASER-004 Empty laser cutter exhaust filter

> He has not yet met the 2-hour volunteer requirement for April.

Example 6: Member queries open tasks for a location

In `#shop-queries`:

> **@alex_johnson:** What tasks are still open in the metalshop?

> **@shop-sergeant-bot: Metalshop — Open Tasks (April 2026)**
>
> | Task | Frequency | Last Done |
> |---|---|---|
> | MSL-METAL-001 — Inspect and clean bandsaw | Weekly | 18 days ago |
> | MSL-METAL-006 — Replace worn grinding wheels | Monthly | 47 days ago |
>
> 2 of 7 metalshop tasks are currently overdue.

Example 7: Steward adds a task

In `#shop-admin`:

> **@sarah_steward:** Add a new task to the metalshop — "Inspect welding equipment for damage", weekly, supervision required

> **@shop-sergeant-bot:** Task created:
>
> - ID: MSL-METAL-008
> - Name: Inspect welding equipment for damage
> - Location: Metalshop
> - Frequency: Weekly
> - Supervisor required: Yes
>
> Google Sheets updated. Generate a new sign-up sheet when ready: `sheet metalshop`

Example 8: Steward updates a task

In `#shop-admin`:

> **@sarah_steward:** Change MSL-METAL-006 frequency from monthly to quarterly

> **@shop-sergeant-bot:** Task updated:
>
> - ID: MSL-METAL-006
> - Field changed: Frequency — Monthly → Quarterly
>
> Google Sheets updated. `#shop-alerts` notified.

Example 9: Steward deletes a task

In `#shop-admin`:

> **@sarah_steward:** Delete MSL-METAL-002 — the plasma cutter has been removed from the shop

> **@shop-sergeant-bot:** Task deleted:
>
> - ID: MSL-METAL-002 — Clean plasma cutter work surface
> - Reason logged: equipment removed
>
> Google Sheets updated. Generate a new sign-up sheet when ready: `sheet metalshop`

Example 10: Steward generates a sign-up sheet

In `#shop-admin`:

> **@sarah_steward:** sheet metalshop

> **@shop-sergeant-bot:** Metalshop sign-up sheet generated.
>
> - Tasks included: 7
> - Generated: 2026-04-13
>
> [Download PDF →]
>
> `#shop-bulletin` notified that a new sheet is ready for printing.

Example 11: Member requests a public location report

In `#shop-queries`:

> **@alex_johnson:** Give me a status report on the laser cutter area

> **@shop-sergeant-bot: Laser Cutter — April 2026 Status**
>
> | Metric | Value |
> |---|---|
> | Tasks defined | 5 |
> | Completed on time | 4 |
> | Overdue | 1 |
> | Completion rate | 80% |
>
> Overdue: MSL-LASER-003 — Calibrate laser alignment (Monthly — last done 38 days ago)

Example 12: Shop Steward triggers overdue reminder blast

In `#shop-admin`:

> **@mike_shopsteward:** Send an overdue reminder now — don't wait for the scheduled one

> **@shop-sergeant-bot:** Overdue reminder blast sent to `#shop-bulletin`. 3 locations have overdue tasks: Metalshop (2), Laser Cutter (1), Wood Shop (1).

Example 13: Shop sergeant views bot errors / audit log

In `#shop-alerts` (Shop Sergeant reads recent entries):

```
[2026-04-13 14:02] TASK_CREATED  MSL-METAL-008 by @sarah_steward
[2026-04-13 14:05] SHEET_GENERATED  Metalshop by @sarah_steward (7 tasks)
[2026-04-13 15:30] TASK_LOGGED  MSL-METAL-003 by @alex_johnson, source=Slack
[2026-04-13 16:00] REMINDER_BLAST  triggered by @mike_shopsteward — 4 overdue tasks ac
[2026-04-13 16:01] BOT_ERROR  Google Sheets write timeout — retry succeeded after 2s
```

Example 14: Shop sergeant requests monthly compliance report *(restricted report)*

In `#shop-admin`:

> **@jeff_sergeant:** Give me the April compliance report

> **@shop-sergeant-bot: April 2026 — Member Compliance Report**

> 38 of 47 active members (81%) met the 2-hour volunteer requirement.
>
> | Status | Count |
> | --- | --- |
> | Compliant (≥ 2 hrs) | 38 |
> | Non-compliant (< 2 hrs) | 7 |
> | No activity recorded | 2 |
>
> Top contributors this month: @alex_johnson (5 tasks), @maria_w (4 tasks), @dave_m (4 tasks) Full report: [View in Google Sheets →]

Example 15: Bot-initiated overdue task reminder *(scheduled, not user-triggered)*

Bot posts to `#shop-bulletin` (automated, weekly):

```
⚠️  Overdue Task Reminder — Metalshop

The following tasks are past due:

• MSL-METAL-001 — Inspect and clean bandsaw (Weekly — last done 18 days ago)
• MSL-METAL-006 — Replace worn grinding wheels (Monthly — last done 47 days ago)

Metalshop steward: @sarah_steward
Sign up to help: [Metalshop Sign-Up Sheet QR →]
```

Example 16: Shop Steward suspends actions for a location

In `#shop-admin`:

> **@mike_shopsteward:** Suspend all actions for the metalshop — we're doing a full equipment audit and don't want any task logging until further notice

> **@shop-sergeant-bot:** ⚠️ Actions suspended — Metalshop
>
> - Scope: location
> - Actions suspended: all
> - Suspended by: @mike_shopsteward
> - Effective: immediately

> Members attempting to log tasks or generate sheets for Metalshop will be told the location is temporarily unavailable. Restore with: `restore metalshop`
>
> `#shop-alerts` notified.

Later, after the audit:

In `#shop-admin`:

> **@mike_shopsteward:** restore metalshop

> **@shop-sergeant-bot:** ✅ Actions restored — Metalshop
>
> - All actions re-enabled for Metalshop
> - Restored by: @mike_shopsteward
>
> `#shop-alerts` notified.

Example 17: Shop Sergeant suspends actions shop-wide

In `#shop-admin`:

> **@jeff_sergeant:** Suspend all bot actions shop-wide — we have a Google Sheets API outage and I don't want failed writes going unnoticed

> **@shop-sergeant-bot:** ⚠️ Actions suspended — Shop-wide (MSL)
>
> - Scope: entire shop
> - Actions suspended: all
> - Suspended by: @jeff_sergeant
> - Effective: immediately
>
> All bot actions are disabled. Members will receive a maintenance notice if they attempt to interact with the bot. Restore with: `restore shop`
>
> `#shop-alerts` notified.

Later, after the outage is resolved:

In `#shop-admin`:

> **@jeff_sergeant:** restore shop

> **@shop-sergeant-bot:** ✅ Actions restored — Shop-wide (MSL)

> - All actions re-enabled across all locations
> - Restored by: @jeff_sergeant
>
> `#shop-alerts` notified.

---

8. Non-Functional Requirements

8.1 Frictionless Member Experience

The system's core design principle is **minimum friction for Members**. Members must be able to record a task completion as easily and quickly as possible — the goal is to remove every possible barrier between (**START**) selecting a task to perform and (**FINISH**) logging the fact that the task has been performed.

- Task logging via the QR method must require **≤ 3 taps** from QR scan to confirmation
- The QR method must not require Members to create an account, log in, or install an app — the entire digital interaction is accessible through the phone's native camera and browser
- The Slack Update method assumes Members have an existing Slack account; no additional account creation is required
- The fallback (writing on the paper sheet) must always remain available alongside the digital methods

8.2 Record Retention and Compliance

- All task completion records must be retained **indefinitely** in Google Sheets
- Records must include: Member name, Task ID, completion date, and capture method
- Data must be exportable from Google Sheets in standard formats (CSV, Excel) for compliance reporting
- These records may be required to support Makersmiths' 501(c)(3) tax-exempt status

8.3 Availability and Resilience

- The Slack bot is the primary interface, but **Google Sheets is the durable store** — if the bot is temporarily unavailable, no data is lost
- The physical sign-up sheet provides a manual fallback for task logging at all times
- Two independent capture paths (QR and Slack) reduce single-point-of-failure risk; if one path is temporarily degraded, members can still log completions via the other

8.4 Access Control

- Slack channel membership is the primary access control mechanism — `#shop-admin` and `#shop-alerts` are restricted channels
- No separate authentication system is required
- The bot must verify actor role before executing restricted actions (task CRUD, compliance reports, etc.)

8.5 Multi-Shop Extensibility

- The data model must support multiple shops from the start (MSL, MSP)
- All task IDs are shop-prefixed (`MSL-`, `MSP-`) to prevent collisions
- The initial release deploys for MSL only, but adding MSP must not require architectural changes

---

9. Success Criteria (KPIs)

The Shop Sergeant is responsible for monitoring the following KPIs to assess system health and process effectiveness.

Member-Facing Metrics

ID	Metric	Definition	Target
M1	Member Compliance Rate	% of active Members who log volunteer some hours in a given month	≥ 50% monthly
M2	Task Completion Rate	% of posted work tasks completed within their scheduled period	≥ 90%
M3	Data Completeness Rate	% of completed tasks with a full record in Google Sheets (name + date + Task ID)	≥ 95%

M1 — Member Compliance Rate

Formula: `(# unique members with ≥ 1 completion record in month) / (# active members) × 100`

Data available:

- ✅ **Numerator** — `completions` sheet in Google Sheets; count distinct `member_name` values where `completion_date` falls in the target month.
- ❌ **Denominator** — the system has no member registry. The total count of "active members" does not exist anywhere in the current design.

Gap & recommendation: Add a `members` sheet to Google Sheets listing all active members (name, Slack handle, join date, status). This sheet becomes the denominator for M1 and enables the compliance report (TR-2, §5.4). Alternatively, pull the active member list from the Slack workspace API, but that ties the KPI to Slack membership which may not perfectly match makerspace membership. The Google Sheets registry is preferred as the authoritative source.

M2 — Task Completion Rate

Formula: `(# tasks with ≥ 1 completion record within their frequency window) / (# total active tasks) × 100`

Frequency windows: weekly = last 7 days; monthly = last 30 days; quarterly = last 90 days.

Data available:

- ✅ **Task catalog** — `tasks` sheet in Google Sheets contains each task's `task_id`, `frequency`, and `location`.
- ✅ **Completion records** — `completions` sheet contains `task_id` and `completion_date` for every logged event.
- ✅ **Calculation** — for each task, find its most recent `completion_date` and compare `today - last_done ≤ frequency_window`. If yes, the task is "completed within schedule."

No data gaps. This KPI is fully computable from existing system data. The Task Reporting pipeline (§5.4) query agent should be able to answer this on demand.

M3 — Data Completeness Rate

Formula: `(# completion records with non-null member_name AND task_id AND completion_date) / (# total completion records) × 100`

Data available:

- ✅ All three required fields (`member_name`, `task_id`, `completion_date`) are required by TC-1 and written to the `completions` sheet on every capture event.

- ✅ For Method 1 (QR-form), all fields are enforced by the form before submission — completeness should be 100%.
- ✅ For Method 2 (Slack Update), the bot confirms task identity with the member before writing (TC-9) — all fields are populated before the record is committed.

Note: This metric only covers records that *entered* the system. It cannot measure tasks that were completed on the shop floor but never recorded at all — that unmeasured gap is exactly the problem M1 and M2 are designed to surface indirectly.

Operational Metrics

ID	Metric	Definition	Target
O1	Steward Adoption Rate	% of Stewards actively using the system to manage tasks	100% within 60 days of launch
O2	Sheet Refresh Latency	Time between a Steward requesting a new sign-up sheet and it being physically posted	≤ 24 hours
O3	Overdue Task Alert Rate	% of overdue tasks that triggered an automated reminder before going unnoticed	100%
O4	Top Contributor Recognition	Monthly top-contributor report generated and posted to `#shop-bulletin` on time	100% monthly

O1 — Steward Adoption Rate

Formula: `(# stewards who issued ≥ 1 bot command in the period) / (# total stewards) × 100`

Data available:

- ✅ **Numerator** — the bot audit log (accessible to Shop Sergeant per §7.3) records every Slack command with the issuing user. Count distinct steward users who issued any `#shop-admin` command.
- ❌ **Denominator** — same gap as M1: the system has no steward registry.

Gap & recommendation: The `members` sheet recommended for M1 should include a `role` column (`member`, `steward`, `shop_steward`, `shop_sergeant`). Filtering that sheet by `role =`

`steward` gives the denominator for O1 at no additional cost.

O2 — Sheet Refresh Latency

Formula: `sheet_posted_timestamp - sheet_requested_timestamp` (per sheet generation event)

Data available:

- ✅ **Request timestamp** — the bot logs the `sheet <location>` command to the audit log with a timestamp (per TS-5, #shop-alerts is notified on every generation). The Slack message timestamp is the request time.
- ✅ **Generation timestamp** — the bot writes the PDF generation completion event to #shop-alerts; this timestamp is also capturable.
- ❌ **Posted-to-wall timestamp** — when a steward physically takes the sheet off the printer and posts it on the shop wall is a manual action with no digital signal. The system cannot observe this.

Gap & recommendation: Redefine O2 as **"time from Slack command to PDF available for download"** — this is fully measurable and is the portion the system controls. The physical posting step is outside the system boundary. Optionally, add a lightweight `!sheet-posted MSL-METAL` Slack command that stewards send after posting the sheet; logging that message would make the full latency measurable, but adds friction and may not be worth it.

O3 — Overdue Task Alert Rate

Formula: `(# overdue tasks for which an APScheduler reminder was sent) / (# overdue tasks detected) × 100`

Data available:

- ✅ **Overdue tasks** — same computation as M2: compare `last_completion_date + frequency_window` to today. Any task past its window is overdue.
- ✅ **Alerts sent** — APScheduler (§4.2) runs the overdue check and posts to `#shop-bulletin`. If each alert post is logged to the audit log with `task_id`, the numerator is derivable.

Requirement: The bot must log each overdue-task reminder event to the audit log (or a dedicated `alerts` sheet) with `task_id` and `alert_timestamp`. Without this log, the numerator is unknowable. This is not currently stated as an explicit requirement — it should be added to §5.4 (TR requirements) or §7.3 (audit log requirements).

O4 — Top Contributor Recognition

Formula: Binary — was the monthly top-contributor report posted to `#shop-bulletin` within the first N days of the month?

Data available:

- ✅ **Report data** — top contributors are derived from the `completions` sheet: count records per `member_name` for the prior month, rank descending.
- ✅ **Execution confirmation** — APScheduler fires the report on a fixed schedule. The Slack message timestamp in `#shop-bulletin` confirms it ran and when.

No data gaps. Both the report content and the on-time check are computable from existing data. The Task Reporting pipeline (TR-6, §5.4) already requires this report. Monitoring O4 requires only checking whether the expected `#shop-bulletin` post appeared within the window.

---

10. Out of Scope

The following are explicitly **not** part of this initial release:

Item	Notes
Makersmiths Purcellville (MSP)	Future release. The system architecture must accommodate MSP without structural changes, but no MSP deployment is planned at this time.
Capture methods not selected	Once a capture method is chosen from the three options in §5.3, the remaining methods are documented for reference only and will not be implemented.
Non-Slack notification channels	Email and SMS notifications are not in scope. Slack is the sole notification channel.
Mobile app	To reduce friction, it is a requirement that there is no need for a native iOS or Android app. The mobile experience is delivered through Slack and a mobile-optimized web form (Method 1 — Single-Sheet QR).

Item	Notes
Real-time dashboards	Reports are delivered on-demand via Slack or on a scheduled basis. No live dashboard is planned. It is expected that real-time status information will be provided via natural language queries.

Appendices

Appendix A: Things Worth Considering

The items below are not requirements — they are ideas that surfaced during design and are worth a deliberate discussion before the relevant phase is implemented. Each may add value, but each also carries cost or complexity that needs to be weighed.

- **Is the §7.3 permission model too open or too tight?** The current permission table grants Stewards the ability to suspend/restore actions at the location scope — a power normally associated with oversight roles — while denying them the ability to query another member's task history. The suspension capability was included so stewards can act quickly in an emergency (equipment removal, safety incident), which is reasonable. However, it also means any steward can unilaterally disable task logging for their location without Shop Steward approval. On the other side, the table shows Stewards cannot trigger an overdue reminder blast, even for their own location — which may frustrate a steward who notices a task is overdue and wants to nudge members immediately. Both edges are worth a deliberate conversation with actual stewards before the bot permission model is implemented.

- **Should the §8.1 requirement that the paper fallback "must always remain available" be dropped?** Section 8.1 lists as a hard requirement that members must always be able to write on the physical sheet as a fallback alongside the digital method. The intent is resilience — if the QR form is unavailable, no completion goes unrecorded. In practice, however, keeping the paper fallback alive indefinitely has a cost: the sign-up sheet must always include hand-entry columns and the system can never fully deprecate the manual process. If the two digital methods prove reliable during the Phase 0 trial and shop Wi-Fi is consistently available, the paper fallback may be more of a crutch than a safety net — one that dilutes the incentive to use the digital flow. The question is whether the fallback is a permanent design principle or a launch-period hedge that should have a planned sunset.

- **Should the mobile experience be a Progressive Web App (PWA)?** The current design calls for a lightweight mobile web form opened by scanning a QR code — no app installation required. A PWA would preserve that zero-install property while adding

capabilities not available to a plain web page: an "Add to Home Screen" prompt so frequent users get a native-looking icon, offline support via a service worker so the form still loads even with spotty shop Wi-Fi, and optional push notifications so members can receive reminders without Slack. The tradeoff is complexity: PWAs require a service worker, a web app manifest, and HTTPS — all achievable but meaningfully more infrastructure than a single-page form. If member re-engagement (repeat task logging, reminders) proves important after the Phase 0 trial, a PWA upgrade is a natural next step. If most members log tasks infrequently and connectivity is reliable, the added complexity is likely not justified.

- **The goal for Members is 2 hr/month of volunteer work but the process doesn't measure hours.** The system currently tracks task completions as binary events — a member either completed a task or did not. There is no notion of how long a task takes, which makes it impossible to tell whether a member who logged two tasks in a month actually contributed 2 hours or 20 minutes. A practical fix is to attach an estimated duration to each task in the data model — a Steward-supplied guess, defaulting to a minimum of 15 minutes for any task that isn't explicitly rated. With per-task estimates in place, the reporting pipeline can sum estimated hours per member per month and flag members who are technically compliant on task count but well short of the 2-hour intent. This also gives Stewards a lever for balancing workload: if a location's tasks are all 15-minute jobs, a compliant member may only be contributing half an hour. Estimated hours are inherently imprecise — the same task might take a new member 45 minutes and an experienced one 10 — but even a rough estimate is more useful than no estimate at all. The question is whether to add a `time` field to the task schema now, while the data model is still being designed, or defer it until compliance reporting becomes a real need.

- **Should we consider Claude Routines as an alternative to APScheduler?** The current design (§4.2) embeds APScheduler inside the bot process to drive all time-based automation: overdue task alerts posted to `#shop-bulletin`, weekly error and availability summaries, monthly compliance summaries, and top-contributor recognition posts. APScheduler is a well-understood Python library — it runs in-process, fires reliably, and adds no per-execution API cost — but it has one structural dependency that is easy to overlook: the bot process must be running continuously for the schedule to execute. If the bot restarts, crashes, or is redeployed, any scheduled job that was due during the outage is silently skipped unless the job store is persisted to a durable backend.

  Claude Routines offer a different model. A Routine is a managed, cloud-scheduled agent: Anthropic's infrastructure fires the agent on a cron schedule, the agent invokes the Claude model with a task prompt, executes any needed tool calls (querying Google Sheets, posting to Slack), and terminates. No persistent bot process is required — each scheduled job is stateless and self-contained. This eliminates the "missed job on restart" failure mode

entirely and removes the operational burden of keeping a long-running process alive around the clock. The agent also has access to Claude's full reasoning capability at execution time, which means a monthly compliance summary could be drafted in natural language rather than templated text — potentially more readable for members, though less predictable for testing.

Appendix B: Work Tasks Data Model

```
opportunities:
  shop:
    name: Makersmiths Leesburg (MSL)
    address: 106 Royal St SW, Leesburg, VA 20175
    steward: John Carter
    area:
      - name: Main Level
        location:
          - name: Metalshop
            steward: Brad Hess
            work_tasks:
              - task: Dust & Wipe Down Machine
                task_id: MSL-METAL-001
                time: 15
                frequency: Weekly
                purpose: Keep shop clean, safe, and easy for all to use
                instructions: Do not use any cleaning solvant. Just a dry or damp clo
                supervision: false
                last_date: NA

              - task: Clear Off & Organize Tabletops
                task_id: MSL-METAL-002
                time: 15
                frequency: Daily
                purpose: Keep the work area as open & free of object for safety
                instructions: NA
                supervision: false
                last_date: NA

              - task: Remove Anything That is Not CNC and Put Away
                task_id: MSL-METAL-003
                time: 15
                frequency: Daily
                purpose: Keep the work area as open & free of object for safety
                instructions: Every CNC / Big Red thing has its place, so place it the
                supervision: false
                last_date: NA
```

```
                - task: Vaccum Floor
                  task_id: MSL-METAL-004
                  time: 15
                  frequency: Daily
                  purpose: Keeping the floor clean is important for safety
                  instructions: NA
                  supervision: false
                  last_date: NA

                - task: Check Dust Collection and Empty if Needed
                  task_id: MSL-METAL-005
                  time: 15
                  frequency: Weekly
                  purpose: Keep shop clean, safe, and easy for all to use.
                  instructions: Pull the handle on the left side and remove the basket.
                  supervision: true
                  last_date: NA

                - task: Empty Shop Vac
                  task_id: MSL-METAL-006
                  time: 15
                  frequency: Bi-Weekly
                  purpose: As the Vac fills up, it losses power and not nearly as effect
                  instructions: Dump its contents in the dumpster in the back of the bu
                  supervision: false
                  last_date: NA

                - task: Empty Saw Dust
                  task_id: MSL-METAL-007
                  time: 15
                  frequency: Bi-Weekly
                  purpose: This helps keep the machine from clogging up while in use.
                  instructions: On the front of the machine, remove the small basket an
                  supervision: true
                  last_date: NA
```

Appendix C: Is it a Pipeline or Workflow?

This project uses the term **pipeline** deliberately — here is why, and when you would reach for *workflow* instead.

Pipeline describes a linear sequence of automated stages where data or artifacts flow in one direction. Each stage transforms its input and passes the result to the next. The metaphor is plumbing: things move through, are processed, and come out the other end.

Workflow describes a broader process that coordinates tasks, people, or systems to accomplish a goal. Workflows can branch on conditions, loop, wait for human input, or run steps in parallel. The metaphor is a flowchart.

	Pipeline	Workflow
Shape	Linear, sequential	Branching, conditional
Actors	Systems / code only	Often includes humans
Triggers	Data arriving	Events, decisions, schedules
State	Stateless stages	Often stateful across steps

Why this project uses *pipeline*: The four named processes — Task Database, Task Sheet, Task Capture, and Task Reporting — are automated data flows with a clear input → output path and no branching logic that requires human decisions mid-stream. That fits *pipeline*.

When *workflow* would apply: If a future feature required a steward to approve a new task before it entered the catalog, or if a member dispute needed escalation, those multi-actor, decision-driven processes would be *workflows*, not pipelines.

Appendix D: Volunteer Opportunity Sign-Up Sheet

Here is an example of the Volunteer Opportunity Sign-Up Sheet

Location:
Metalshop
Steward: Brad Hess

Volunteer Opportunities

MAKERSMITHS
INNOVATION FORGE

Task Name	Last Done	Frequency	Member Name / Date	Member Name / Date	Member Name / Date
Dust & Wipe Down Machine Do not use any cleaning solvant. Just a dry or damp cloth should be used. MSL-METAL-001	NA	Weekly			
Clear Off & Organize Tabletops MSL-METAL-002	NA	Daily			
Remove Anything That is Not CNC and Put Away Every CNC / Big Red thing has its place, so place it there. MSL-METAL-003	NA	Daily			
Vaccum Floor MSL-METAL-004	NA	Daily			
Check Dust Collection and Empty if Needed Pull the handle on the left side and remove the basket. Dump its contents in the dumpster in the back of the building. MSL-METAL-005	NA	Weekly			
Empty Shop Vac Dump its contents in the dumpster in the back of the building. MSL-METAL-006	NA	Bi-Weekly			
Empty Saw Dust On the front of the machine, remove the small basket and put in the dumpster. MSL-METAL-007	NA	Bi-Weekly			

Questions concerning the design & use of this form should be sent to Jeff Irland (xxx)